Introduction

Bootloader role

- The bootloader is a piece of code responsible for
  - Basic hardware initialization
  - Loading of an application binary, usually an operating system kernel, from flash storage, from the network, or from another type of non-volatile storage.
  - Possibly decompression of the application binary
  - Execution of the application
- Besides these basic functions, most bootloaders provide a shell or menu
  - Menu to select the operating system to load
  - Shell with commands to load data from storage or network, inspect memory, perform hardware testing/diagnostics
- The first piece of code running by the processor that can be modified by us developers.

Booting on x86 platforms

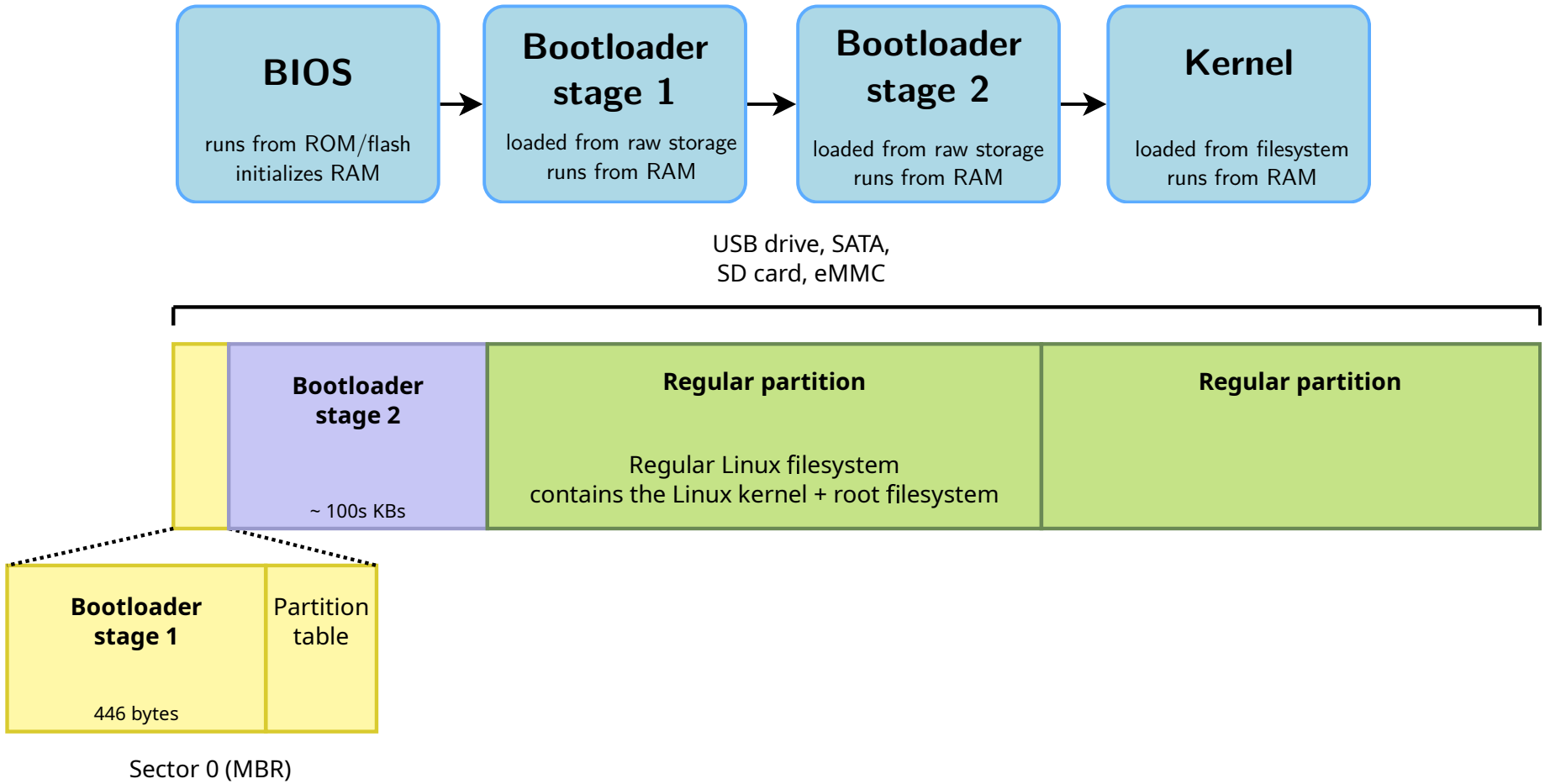
Legacy BIOS booting (1)

- x86 platforms shipped before 2005-2006 include a firmware called *BIOS*
  - BIOS = Basic Input Output System
  - Part of the hardware platform, closed-source, rarely modifiable
  - Implements the booting process
  - Provides runtime services that can be invoked - not commonly used
  - Stored in some flash memory, outside of regular user-accessible storage devices
- To be bootable, the first sector of a storage device is “special”
  - MBR = Master Boot Record
  - Contains the partition table
  - Contains up to 446 bytes of bootloader code, loaded into RAM and executed
  - The BIOS is responsible for the RAM initialization
- https://en.wikipedia.org/wiki/BIOS

Legacy BIOS booting (2)

- Due to the limitation in size of the bootloader, bootloaders are split into two stages
  - Stage 1, which fits within the 446 bytes constraint
  - Stage 2, which is loaded by stage 1, and can therefore be bigger
- Stage 2 is typically stored outside of any filesystem, at a fixed offset $arrow.r$_simpler to load by stage 1
- Stage 2 generally has filesystem support, so it can load the kernel image from a filesystem

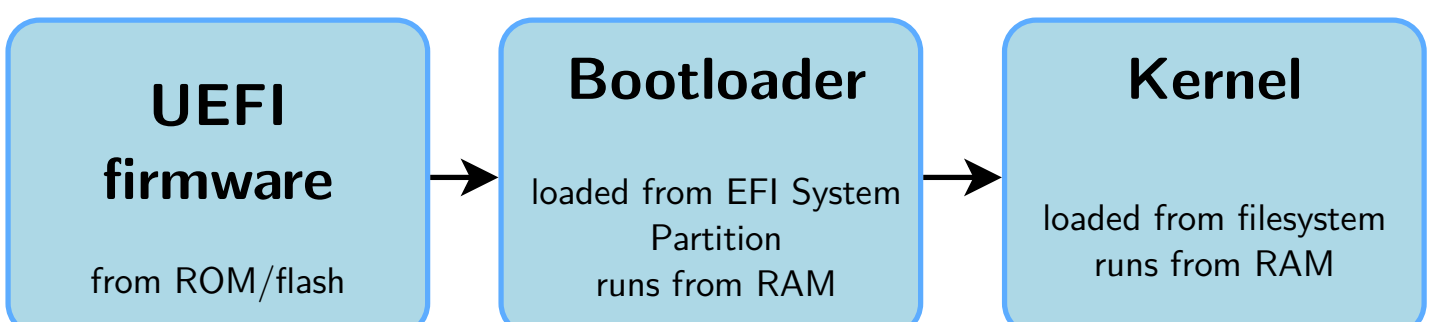
Legacy BIOS booting: sequence and storage



UEFI booting

- Starting from 2005-2006, UEFI is the new firmware interface on x86 platforms
  - Unified Extensible Firmware Interface
  - Describes the interface between the operating system and the firmware
  - Firmware in charge of booting
  - Firmware also provides runtime services to the operating system
  - Stored in some flash memory, outside of regular user-accessible storage devices
- Loads EFI binaries from the *EFI System Partition*
  - Generally a bootloader
  - Can also be directly the Linux kernel, with an *EFI Boot Stub*
- Special partition, formatted with the *FAT* filesystem
  - MBR: identified by type `0xEF`
  - GPT: identified with a specific *globally unique identifier*
- File `/efi/boot/bootx32.efi`, `/efi/boot/bootx64.efi`
- `https://en.wikipedia.org/wiki/UEFI`

UEFI booting: sequence and storage



UEFI booting: sequence and storage

USB drive, SATA,
SD card, eMMC, etc.

ACPI

- Advanced Configuration and Power Interface
- *Open standard that operating systems can use to discover and configure computer hardware components, to perform power management, to perform auto configuration, and to perform status monitoring*
- *Tables* with descriptions of the hardware that cannot be dynamically discovered at runtime
- Tables provided by the firmware (UEFI or legacy) and used by the operating system (Linux kernel in our case)
- https://en.wikipedia.org/wiki/Advanced_Configuration_and_Power_Interface

UEFI and ACPI on ARM

- Historically UEFI and ACPI are technologies coming from the Intel/x86 world
- ARM is also pushing for the adoption of UEFI and ACPI as part of its *ARM System Ready* certification
  - Mainly for servers/workstations SoCs
  - Does not impact embedded SoCs
  - Currently not common in embedded Linux projects on ARM
  - https://www.arm.com/architecture/system-architectures/systemready-certification-program
- Also some on-going effort to use UEFI on RISC-V, but not the de-facto standard
- When an embedded platform uses UEFI $arrow.r$_its booting process is similar to an *x86* platform

Booting on embedded platforms

Booting on embedded platforms: ROM code

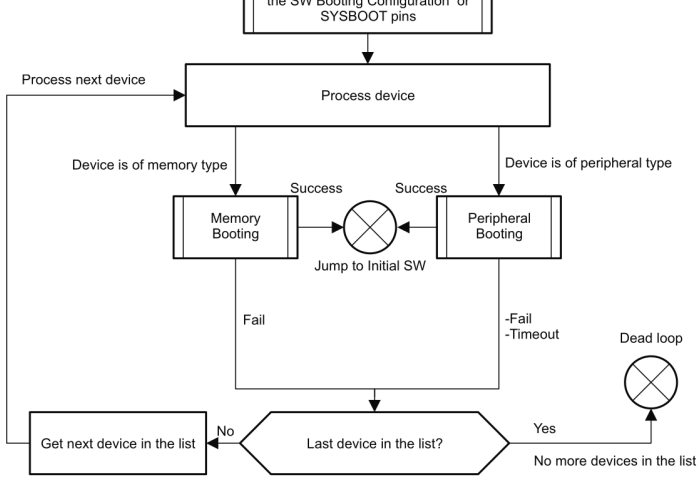
- Most embedded processors include a **ROM code** that implements the initial step of the boot process
- The ROM code is written by the processor vendor and directly built into the processor
  - Cannot be changed or updated
  - Its behavior is described in the processor datasheet
- Responsible for finding a suitable bootloader, loading it and running it
  - From NAND/NOR flash, from USB, from SD card, from eMMC, etc.
  - Well defined location/format
- *Generally* runs with the external RAM not initialized, so it can only load the bootloader into an internal SRAM
  - Limited size of the bootloader, due to the size of the SRAM
  - Forces the boot process to be split in two steps: first stage bootloader (small, runs from SRAM, initializes external DRAM), second stage bootloader (larger, runs from external DRAM)

Booting on STM32MP1: datasheet

l)	Wait incoming conn – USART2/3/6 and – USB High-Speed
n(3)	Serial NOR-Flash o
	eMMC™ on SDMM
	SLC NAND-Flash o
(No	Used to get debug a

Booting on AM335x (32 bit BeagleBone): datasheet

agleBone): datasheet



Once the booting device list is set up, the booting routine examines the devices enumerated in the list sequentially and either executes the memory booting or peripheral booting procedure depending on the booting device type. The memory booting procedure is executed when the booting device type is one of NOR, NAND, MMC or SPI-EEPROM. The peripheral booting is executed when the booting device type is Ethernet, USB or UART.

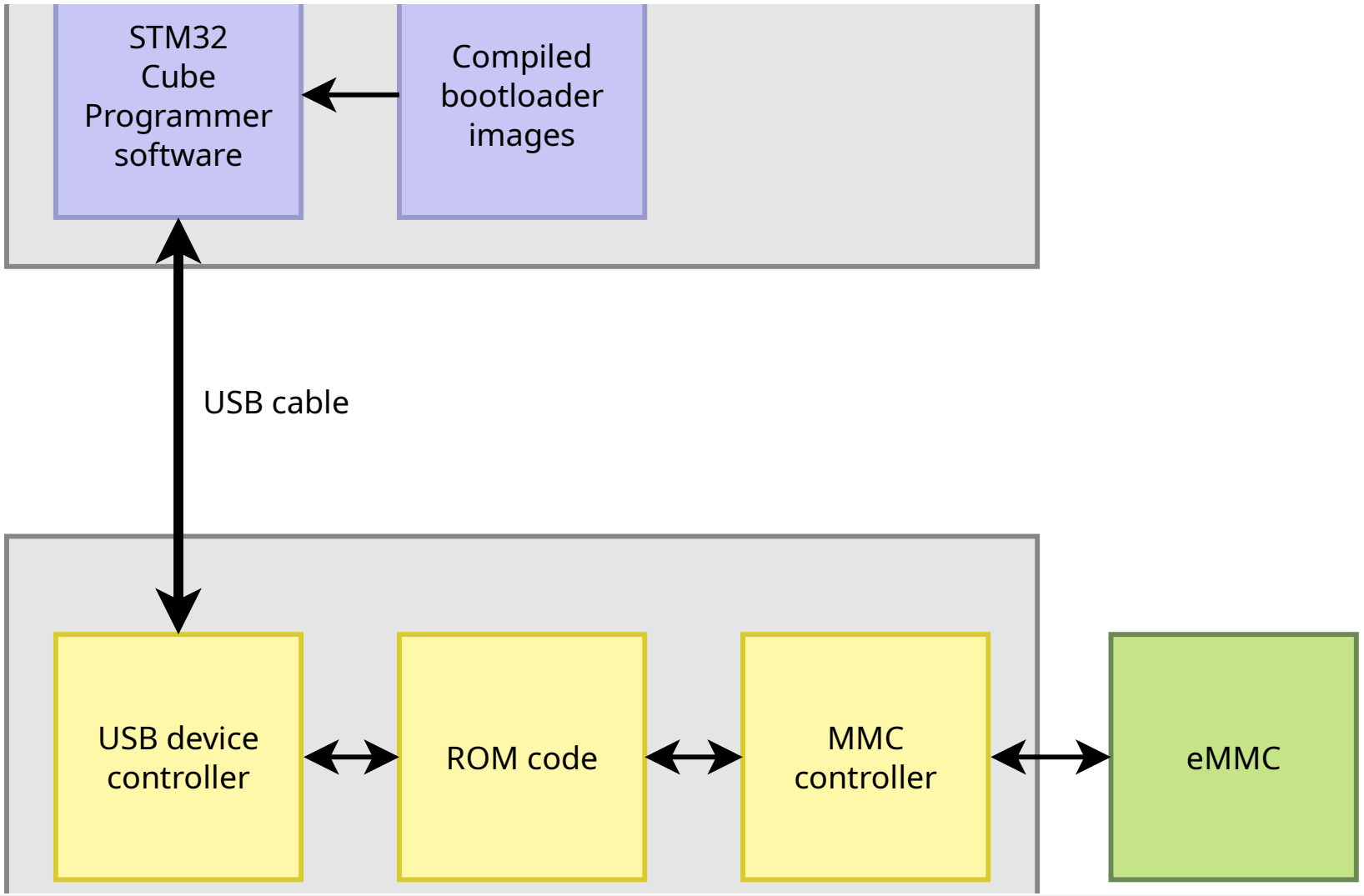
Source: https://www.mouser.com/pdfdocs/spruh73h.pdf, chapter 26

Two stage booting sequence

ROM code recovery mechanism

- Most ROM code also provide some sort of *recovery* mechanism, allowing to flash a board with no bootloader or a broken one, usually with a vendor-specific protocol over UART or USB.
- Often allows to push a new bootloader into RAM, making it possible to reflash the bootloader.
- Vendor-specific tools to run on the workstation
  - STM32MP1: `STM32 Cube Programmer`
  - NXP i.MX: `uuu`
  - Microchip AT91/SAM: `SAM-BA`
  - Allwinner: `sunxi-fel`
  - Some open-source, some proprietary
- Snagboot: new vendor agnostic tool replacing the above ones: `https://github.com/bootlin/snagboot`

ROM code recovery mechanism



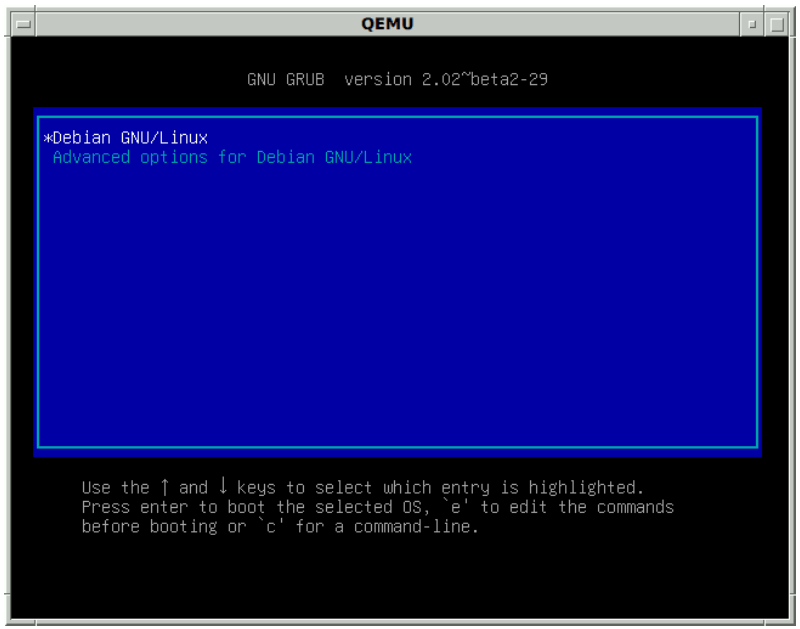
Bootloaders

GRUB

- *Grand Unified Bootloader*, from the GNU project
- De-facto standard in most Linux distributions for x86 platforms
- Supports x86 legacy and UEFI systems
- Can read many filesystem formats to load the kernel image, modules and configuration
- Provides a menu and powerful shell with various commands
- Can load kernel images over the network
- Also supports ARM, ARM64, RISC-V, PowerPC, but less popular than other bootloaders on those platforms

GRUB

- https://www.gnu.org/software/grub/
- https://en.wikipedia.org/wiki/GNU_GRUB



Syslinux

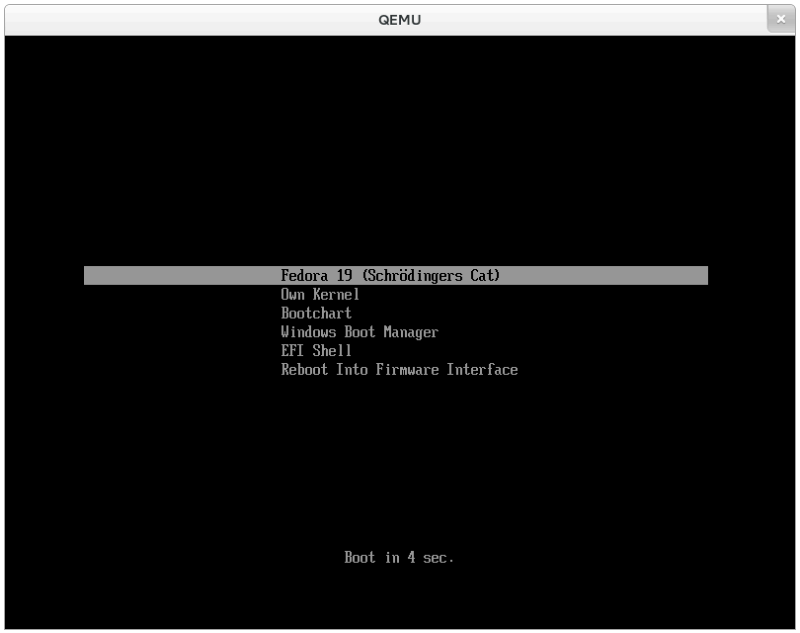
- For network and removable media booting (USB key, SD card, CD-ROM)
- `syslinux`: booting from FAT filesystem
- `pxelinux`: booting from the network
- `isolinux`: booting from CD-ROM
- `extlinux`: booting from numerous filesystem types
- A bit rustic to build and configure, not very actively maintained, but still useful for specific use-cases
- https://wiki.syslinux.org/
- https://kernel.org/pub/linux/utils/boot/syslinux/


SYSLINUX

systemd-boot

- Simple UEFI boot manager
- Useful alternative to GRUB for UEFI systems: simpler than GRUB
- Configured using files stored in the *EFI System Partition*
- Part of the *systemd* project, even though obviously distinct from *systemd* itself
  - See our slides later in this course for more details on *systemd*
- https://www.freedesktop.org/wiki/Software/systemd/systemd-boot/

systemd-boot

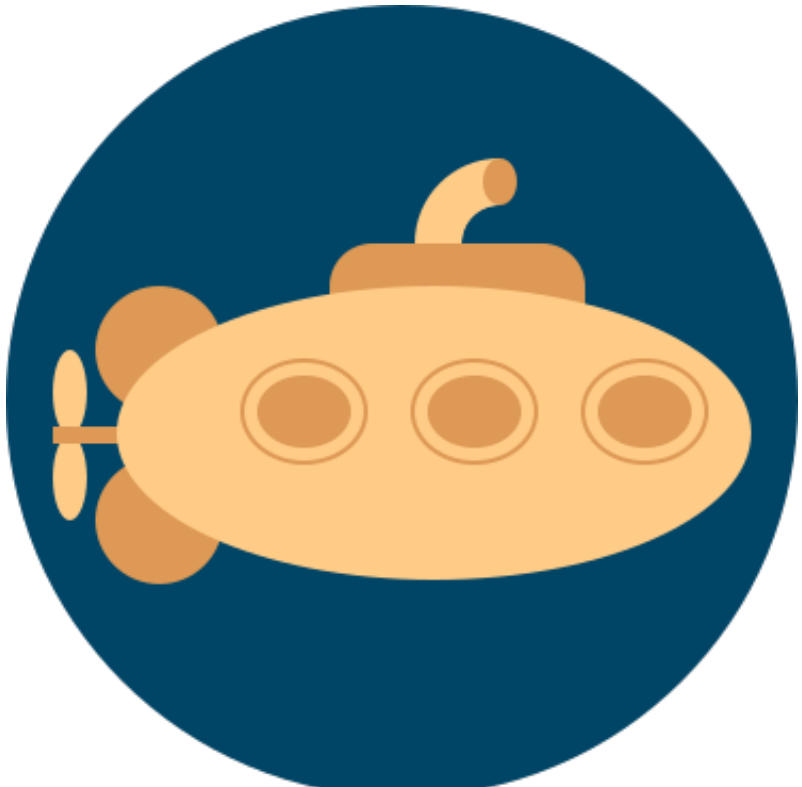


shim

- Minimal UEFI bootloader
- Mainly used in secure boot scenario: it is signed by Microsoft and therefore successfully verified by UEFI firmware in the field
- Allows to chainload another bootloader (GRUB) or directly the Linux kernel, with signature checking
- https://github.com/rhboot/shim

U-Boot

- The de-facto standard and most widely used bootloader on embedded architectures: ARM, ARM64, RISC-V, PowerPC, MIPS, and more.
- Also supports x86 with UEFI firmware.
- Very likely the one provided by your SoC vendor, SoM vendor or board vendor for your hardware.
- We will study it in detail in the next section, and use it in all practical labs of this course.
- https://www.denx.de/wiki/U-Boot



U-Boot

Barebox

- Another bootloader for most embedded CPU architectures: ARM/ ARM64, MIPS, PowerPC, RISC-V, x86, etc.
- Initially developed as an alternative to U-Boot to address some U-Boot shortcomings
  - *kconfig* for the configuration like the Linux kernel
  - well-defined *device model* internally
  - More Linux-style shell interface
  - Cleaner code base
- Actively maintained and developed, but
  - Less widely used than U-Boot

Barebox

- Less platform support than in U-Boot

- https://www.barebox.org/
- Talk *barebox Bells and Whistles*, by Ahmad Fatoum, ELCE 2020, video and slides

Trusted firmware

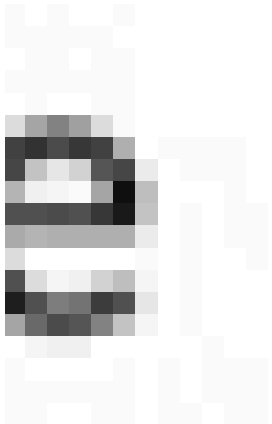
Concept

- Traditionally, bootloaders are only used during the booting process
  - Bootloader loads operating system, jumps to it, and is discarded
- Modern SoCs have advanced security mechanisms that require running some sort of *trusted firmware*
- This firmware is loaded by the bootloader, or part of the boot chain itself
- This *trusted firmware* **stays resident** after control has been passed to the OS
  - It is stored in a dedicated portion of the DDR, or some specific SRAM, inaccessible from the OS
  - It provides services to the OS, which the OS cannot perform directly
  - Can also be responsible for running a secure OS alongside the regular OS (Linux in our case)

ARM

- Modern ARMv7 and ARMv8 processors have
  - 4 privilege levels (*Exception Levels*)
    - EL3, the most privileged, runs secure firmware
    - EL2, typically used by hypervisors, for virtualization
    - EL1, used to run the Linux kernel
    - EL0, used to run Linux user-space applications
  - 2 *worlds*
    - Normal world, used to run a general purpose OS, like Linux
    - Secure world, to run a separate, isolated, secure operating system and applications. Also called *TrustZone* by ARM.
- EL3 only exists in the secure world
- EL2 exists in both secure and normal worlds since ARMv8.4, before that EL2 was only in the normal world
- EL1 and EL0 exist in both secure and normal worlds

ARM exception levels and worlds



ARM exception levels and worlds

Source: ARM documentation

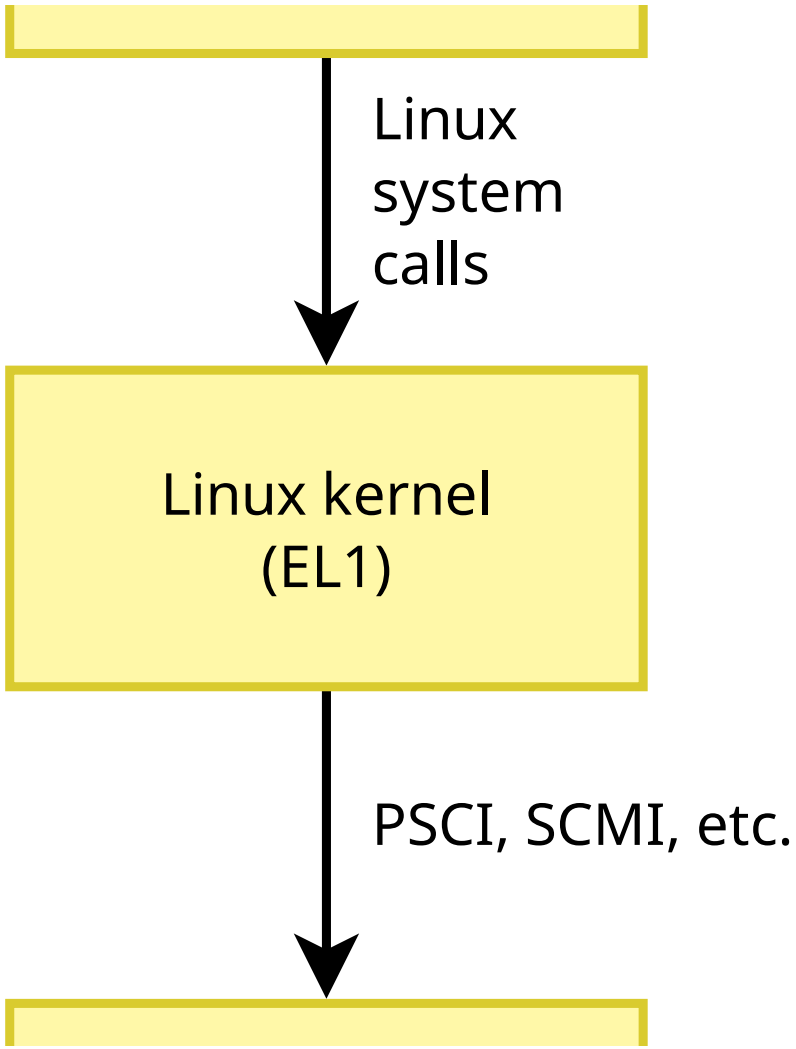
Interfaces with secure firmware

- Standardized by ARM
- Services
  - implemented by the secure firmware
  - called by the operating system
- Prevents the operating system running in normal world from directly accessing critical hardware resources
- PSCI, Power State Coordination Interface
  - Power management related: turn CPUs on/off, CPU idle state, platform shutdown/reset
- SCMI, System Control and Management Interface
  - Power domain, clocks, sensor, performance

Interfaces with secure firmware

- Secure firmware implementing these interfaces is
  - Mandatory to run Linux on ARMv8
  - Mandatory to run Linux on some ARMv7 platforms, but not all

Interfaces with secure firmware



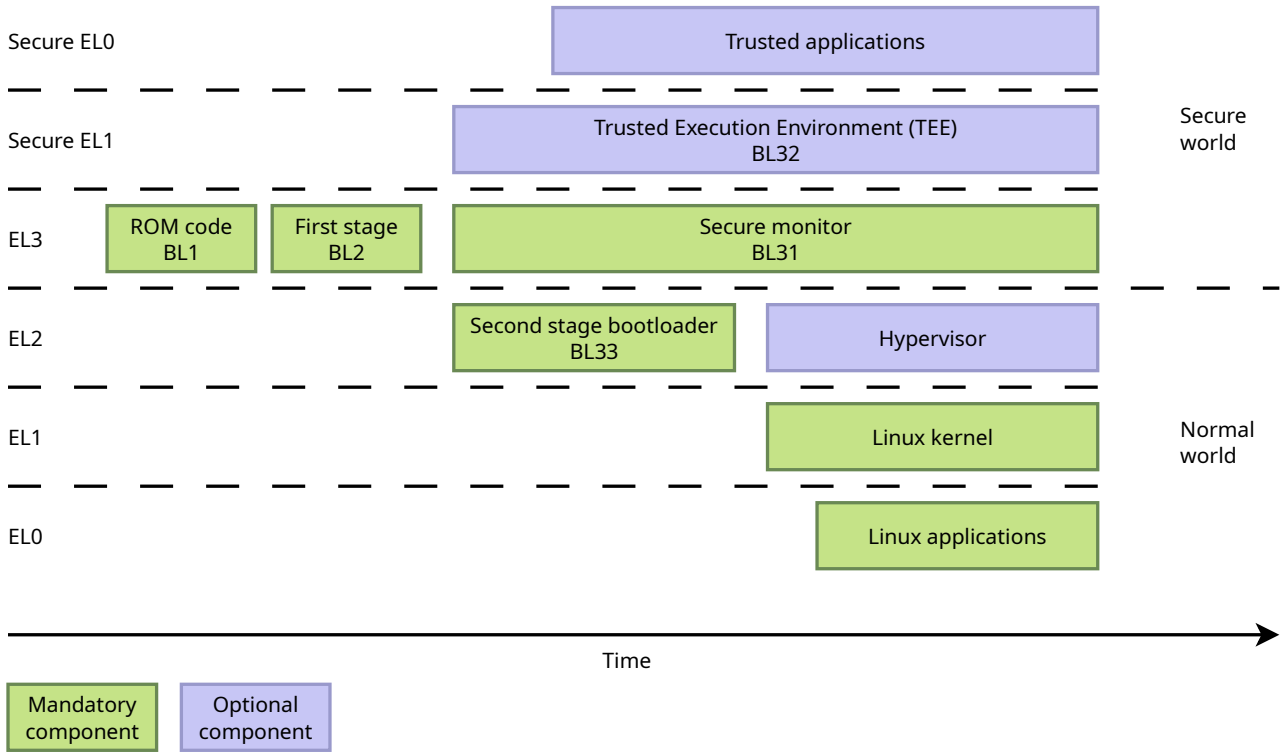
TF-A

- *Trusted Firmware-A (TF-A) provides a reference implementation of secure world software for Armv7-A and Armv8-A, including a Secure Monitor executing at Exception Level 3 (EL3)*
- Formerly known as *ATF*, for ARM Trusted Firmware
- Implements the various standard interfaces that operating systems need from the secure firmware
- Has drivers for the hardware blocks that are not accessed directly by Linux
- Needs to be ported for each SoC
- Depending on the platform, may also need to be ported per board: DDR initialization
- Used on the vast majority of ARMv8 platforms, and on a few recent ARMv7 platforms
- https://www.trustedfirmware.org/projects/tf-a/

Trusted OS, OP-TEE

- A trusted operating system can run in the *secure world*, also called *Trusted Execution Environment* or *TEE*
- Hardware partitioning between *secure world* and *normal world*
  - Some hardware resources only available in the *secure world*, by the trusted OS
- Allows to run trusted applications/services
  - isolated from Linux
  - can provide services to Linux applications
- Most common open-source implementation: *OP-TEE*
  - Supported by most silicon vendors
  - https://www.op-tee.org/

ARM: summary

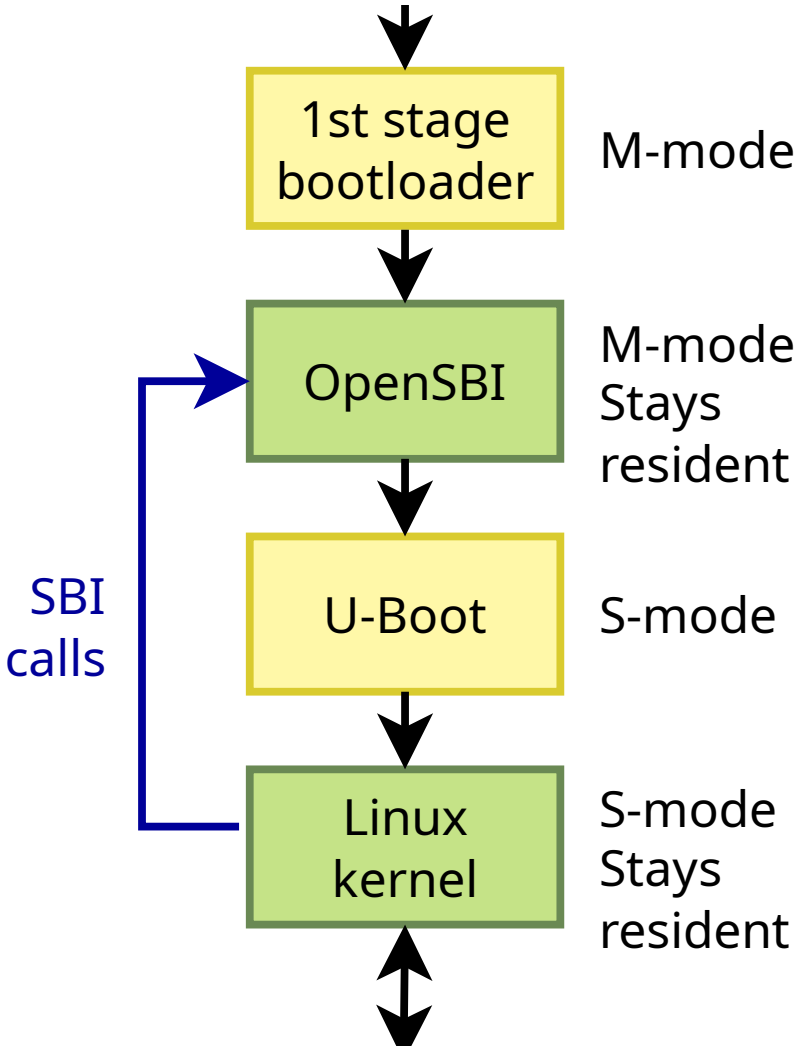


Largely inspired from *Ahmad Fatoum* presentation *From Reset Vector to Kernel*, `slides`, `video` See also `details about the ARM terms: BL1, BL2…`

RISC-V

- Linux-class RISC-V processors have several privilege levels
  - M-mode: machine mode
  - S-mode: level at which the Linux kernel runs
  - U-mode: level at which Linux user-space applications run
- Some specific HW resources are not accessible in S-mode
- A more privileged firmware runs in M-mode
- RISC-V has defined SBI, *Supervisor Binary Interface*
  - Standardized interface between the OS and the firmware

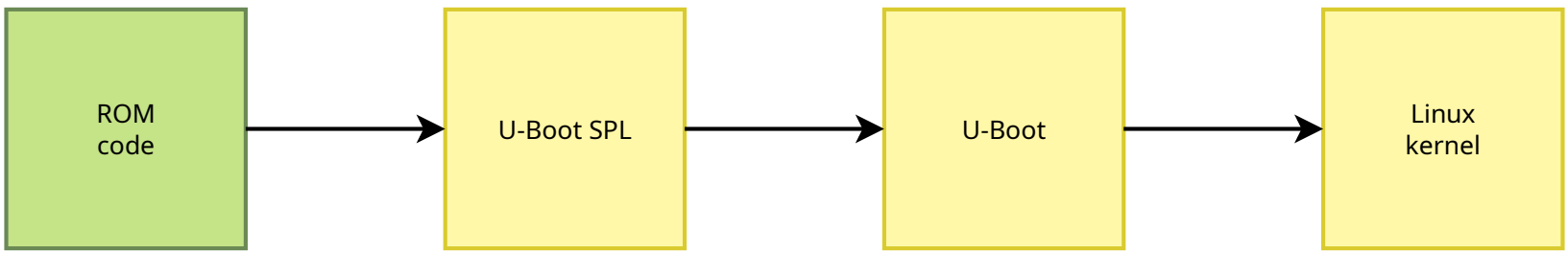
  - https://github.com/riscv-non-isa/riscv-sbi-doc
- OpenSBI is a reference, open-source implementation of SBI
  - https://github.com/riscv-software-src/opensbi


1st stage bootloader
M-mode
OpenSBI
M-mode Stays resident
U-Boot
S-mode
SBI calls
Linux kernel
S-mode Stays resident

RISC-V

Example boot sequences on ARM

TI AM335x (32 bit BeagleBone): ARMv7



Stored in ROM

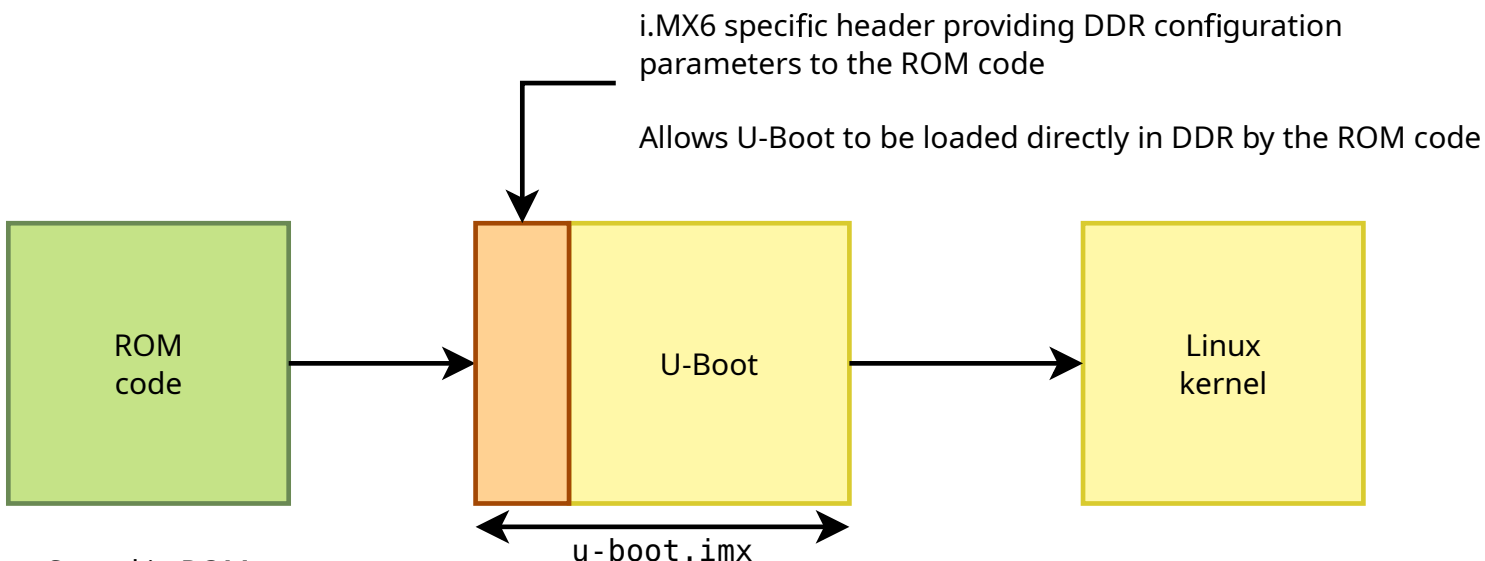
Loaded from a file called MLO in a FAT filesystem in the first bootable partition

Runs from SRAM

Loaded from a file called u-boot.img in a FAT filesystem

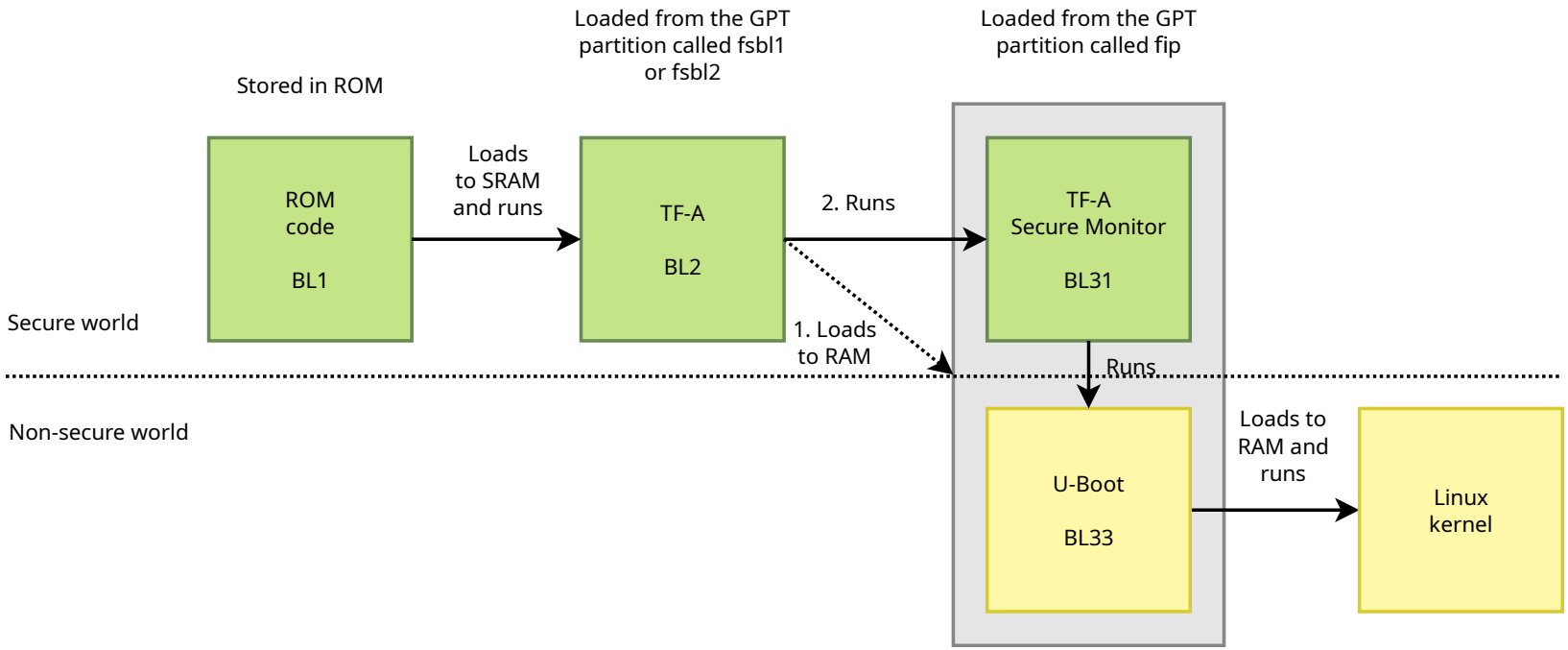
Runs from RAM

NXP i.MX6: ARMv7



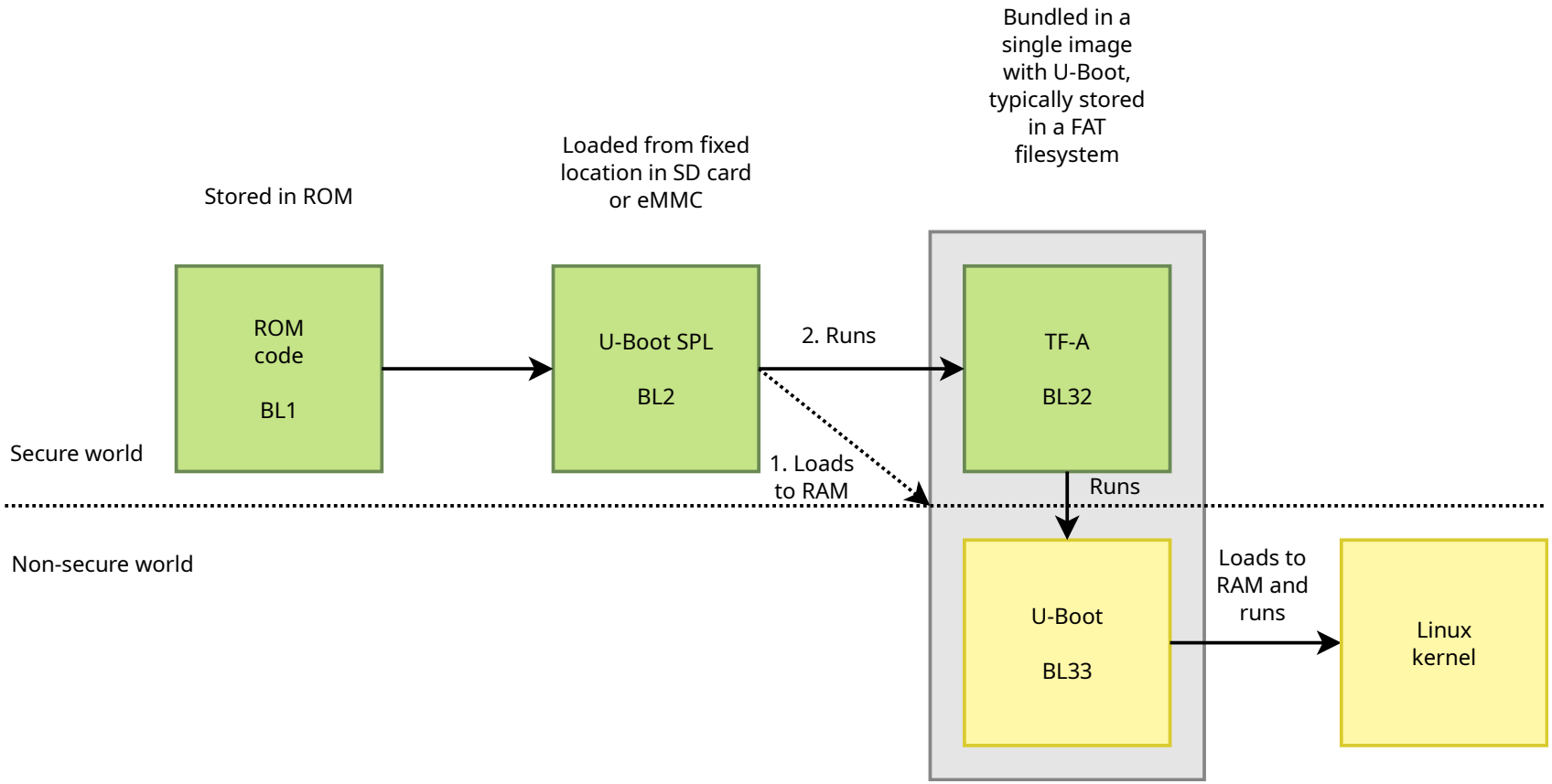
Note: this diagram shows one possible boot flow on NXP i.MX6, but it is also possible to use the U-Boot SPL $arrow.r$_U-Boot boot flow on i.MX6.

STM32MP1: ARMv7

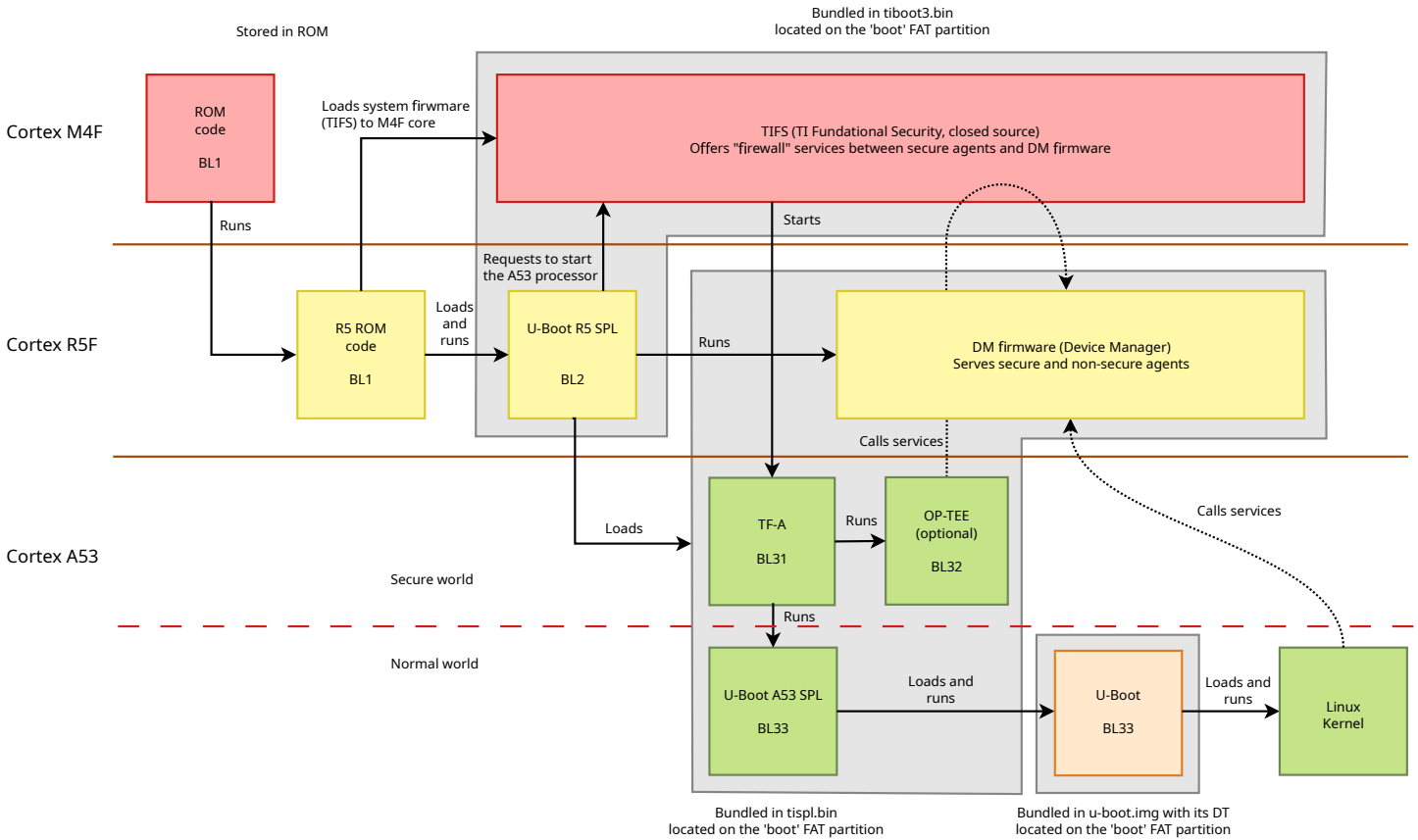


Note: booting with U-Boot SPL and U-Boot is also possible.

Allwinner ARMv8 cores



TI AM62x (BeaglePlay): ARMv7 and ARMv8 cores



See https://u-boot.readthedocs.io/en/latest/board/ti/am62x_sk.html for details.